

IsabelleGym Async Client Documentation

`client.async_client`

This document describes the usage of the asynchronous Python client for the Isabelle server API. It is written in package-reference style and focuses on client lifecycle, session ownership, proof interaction, and big-step verification workflows.

Contents

1	Purpose and Scope	3
2	Public API Surface	3
3	Usage Model	3
3.1	Client Lifetime	3
3.2	Session Ownership	4
3.3	Two Supported Workflows	4
4	Reference: Helper Function	4
4.1	<code>extract_theory_name</code>	4
5	Reference: IsabelleGymAsyncClient	5
5.1	Constructor	5
5.2	Context Manager Methods	5
5.2.1	<code>__aenter__</code> and <code>__aexit__</code>	5
5.2.2	<code>close</code>	5
6	Reference: Server and Session Operations	6
6.1	<code>health</code>	6
6.2	<code>create_session</code>	6
6.3	<code>acquire_session</code>	7
6.4	<code>release_session</code>	8
6.5	<code>close_session</code>	9
7	Reference: Stateful Proof Operations	9
7.1	<code>enter_theory</code>	9
7.2	<code>execute_command</code>	10
8	Reference: Big-Step Verification	11
8.1	General Remarks	11
8.2	<code>verify_bigstep_text</code>	11
8.3	<code>verify_bigstep_file</code>	12

9	Recommended Usage Patterns	13
9.1	Pattern A: Dedicated Session for Stateful Work	13
9.2	Pattern B: Pooled Session for Reusable Capacity	13
9.3	Pattern C: Whole-Theory Verification	14
10	Error and Exception Semantics	14
10.1	HTTP-level failures	14
10.2	Big-step methods	14
10.3	Transport-level failures	15
10.4	Local validation failures	15
11	Return-Type Summary	15
12	Operational Notes	15
13	Minimal End-to-End Example	16
14	Release Note for Package Consumers	16

1 Purpose and Scope

The module `client.async_client` provides an asynchronous Python interface to the IsabelleGym HTTP API. It is designed for callers who want to manage server-backed Isabelle sessions, execute commands incrementally, switch theories, and submit complete theory texts for big-step verification.

This client is a thin, explicit wrapper over the REST interface. It does not hide server ownership semantics. In particular, session access is lease-based: stateful operations must be issued with the lease token returned by session creation or acquisition endpoints.

2 Public API Surface

The module exposes the following public entry points:

- `extract_theory_name(text: str) -> Optional[str]`
- `class IsabelleGymAsyncClient`

The client class provides the following primary operations:

- server health inspection
- dedicated session creation
- pooled session acquisition
- leased session release or closure
- theory entry
- command execution
- big-step verification from an in-memory theory text
- big-step verification from a theory file

3 Usage Model

3.1 Client Lifetime

`IsabelleGymAsyncClient` owns a reusable `httpx.AsyncClient`. The recommended pattern is to use the class as an asynchronous context manager so that the underlying HTTP transport is closed deterministically.

Listing 1: Recommended client lifetime pattern

```
from client.async_client import IsabelleGymAsyncClient

async def main() -> None:
    async with IsabelleGymAsyncClient("http://localhost:8000") as client:
        health = await client.health()
        print(health)
```

It may also be instantiated directly and later closed by calling `await client.close()`.

3.2 Session Ownership

The server associates a lease with each created or acquired session. The client reflects that model directly:

- `create_session(...)` returns a new dedicated session together with a `lease_id`.
- `acquire_session(...)` returns either a reused pooled session or a newly created one, again with a `lease_id`.
- stateful methods such as `execute_command`, `enter_theory`, `release_session`, and `close_session` require the lease identifier when operating on leased sessions.

The lease value is sent through the `X-Lease-Id` request header.

3.3 Two Supported Workflows

Dedicated session workflow. Use `create_session` when the caller wants an isolated backend session whose lifetime is controlled explicitly.

Pooled session workflow. Use `acquire_session` when the caller wants a matching reusable session from the server pool. When the work is complete, return the lease with `release_session` rather than destroying the backend.

4 Reference: Helper Function

4.1 `extract_theory_name`

Signature

```
def extract_theory_name(text: str) -> Optional[str]:
    ...
```

Behavior

This helper scans Isabelle theory text and extracts the declared theory name from a `theory ...` header. It supports both quoted and unquoted names. If no theory declaration is found, the function returns `None`.

Usage example

```
from client.async_client import extract_theory_name

theory_text = """
theory Scratch
imports Main
begin
lemma \"A  $\longrightarrow$  A\" by simp
```

```

end
"""

name = extract_theory_name(theory_text)
print(name)  # Scratch

```

5 Reference: IsabelleGymAsyncClient

5.1 Constructor

Signature

```

IsabelleGymAsyncClient(
    base_url: str = "http://localhost:8000",
    timeout: float = 30.0,
)

```

Parameters

Parameter	Type	Meaning
<code>base_url</code>	<code>str</code>	Base URL of the IsabelleGym server. Trailing slashes are normalized away.
<code>timeout</code>	<code>float</code>	Default request timeout, in seconds, used when a method does not supply a method-specific override.

Notes

- The client internally creates `httpx.AsyncClient(base_url=..., timeout=...)`.
- Methods that accept an explicit timeout may override the default value for a single request.

5.2 Context Manager Methods

5.2.1 `__aenter__` and `__aexit__`

The class supports `async with`. Exiting the context closes the underlying HTTP client.

5.2.2 `close`

Signature

```

async def close(self) -> None:
    ...

```

Closes the underlying `httpx.AsyncClient`. Once closed, the client should not be reused.

6 Reference: Server and Session Operations

6.1 health

Signature

```
async def health(self) -> dict[str, Any]:  
    ...
```

Behavior

Fetches the server root endpoint and returns the decoded JSON payload. The method calls `response.raise_for_status()`, so non-success responses propagate as `httpx.HTTPStatusError`.

Typical response fields

- service
- version
- status
- active_sessions
- busy_sessions
- max_pool_size
- timestamp

Example

```
async with IsabelleGymAsyncClient() as client:  
    info = await client.health()  
    print(info["status"])
```

6.2 create_session

Signature

```
async def create_session(  
    self,  
    theories: list[str] | None = None,  
    field: str | None = "HOL",  
) -> dict[str, Any]:  
    ...
```

Behavior

Creates a new leased session on the server. The response JSON is returned as a Python dictionary. The method raises on non-success status codes.

Inputs

- **theories**: optional dependency theory list supplied to the server.
- **field**: logical field, defaulting to "HOL". The client sends the field when it is not `None`.

Important return fields

- `session_id`
- `lease_id`
- `created_at`
- `theories`
- `status`

Example

```

async with IsabelleGymAsyncClient() as client:
    session = await client.create_session(theories=["Main"], field="HOL")
    session_id = session["session_id"]
    lease_id = session["lease_id"]

    try:
        result = await client.execute_command(
            session_id,
            'lemma "A <longrightrightarrow> A" by simp',
            lease_id=lease_id,
        )
        print(result["success"])
    finally:
        await client.close_session(session_id, lease_id=lease_id)

```

6.3 acquire_session

Signature

```

async def acquire_session(
    self,
    theories: list[str] | None = None,
    field: str | None = "HOL",
    reuse_dirty: bool = True,
) -> dict[str, Any]:
    ...

```

Behavior

Requests a leased session from the server pool. The server may return a previously created compatible session or create a new session on demand. The returned payload is decoded JSON. Non-success responses raise an exception.

Parameters

Parameter	Type	Meaning
<code>theories</code>	optional list	Requested dependency theories for matching or creation.
<code>field</code>	optional str	Requested logical field, typically <code>HOL</code> .
<code>reuse_dirty</code>	bool	If <code>True</code> , sessions with prior command history may be reused when otherwise compatible. If <code>False</code> , only clean sessions may be reused.

Important return fields

All fields from `create_session` are present, plus:

- `reused`: `True` if an existing session was returned, `False` if a new session was created.

Example

```

async with IsabelleGymAsyncClient() as client:
    lease = await client.acquire_session(
        theories=["Main"],
        field="HOL",
        reuse_dirty=False,
    )
    session_id = lease["session_id"]
    lease_id = lease["lease_id"]

    try:
        state = await client.execute_command(
            session_id,
            'lemma "A \<longrightrightarrow> A"',
            lease_id=lease_id,
        )
        print(state["subgoals"])
    finally:
        await client.release_session(session_id, lease_id=lease_id)

```

6.4 release_session

Signature

```

async def release_session(
    self,
    session_id: str,
    *,
    lease_id: str | None = None,
) -> dict[str, Any]:
    ...

```

Behavior

Releases the lease on a pooled session without destroying the backend. This is the normal completion path after `acquire_session`. If the response contains a JSON body, the decoded body is returned; otherwise an empty dictionary is returned. The method raises on non-success status codes.

Example

```
await client.release_session(session_id, lease_id=lease_id)
```

6.5 close_session

Signature

```
async def close_session(  
    self,  
    session_id: str,  
    *,  
    lease_id: str | None = None,  
) -> dict[str, Any]:  
    ...
```

Behavior

Destroys the backend session. This is the normal completion path after `create_session`. As with `release_session`, the method returns the decoded JSON body when one is present and otherwise returns an empty dictionary.

Example

```
await client.close_session(session_id, lease_id=lease_id)
```

7 Reference: Stateful Proof Operations

7.1 enter_theory

Signature

```
async def enter_theory(  
    self,  
    session_id: str,  
    theory_name: str,  
    *,  
    lease_id: str | None = None,  
) -> dict[str, Any]:  
    ...
```

Behavior

Requests that the session switch into a named theory already known to the backend environment. The method returns the decoded server payload when present and otherwise an empty dictionary. The call raises on non-success status codes.

Example

```
async with IsabelleGymAsyncClient() as client:  
    session = await client.create_session(theories=["Main"])
```

```

session_id = session["session_id"]
lease_id = session["lease_id"]

try:
    await client.enter_theory(session_id, "Main", lease_id=lease_id)
    result = await client.execute_command(
        session_id,
        'lemma "True" by simp',
        lease_id=lease_id,
    )
    print(result["output"])
finally:
    await client.close_session(session_id, lease_id=lease_id)

```

7.2 execute_command

Signature

```

async def execute_command(
    self,
    session_id: str,
    command: str,
    timeout: float | None = None,
    *,
    lease_id: str | None = None,
) -> dict[str, Any]:
    ...

```

Behavior

Executes a single Isabelle command in an existing session. The method posts both the command text and an effective timeout to the server. The returned value is a decoded JSON response and non-success HTTP statuses raise immediately.

Representative response fields

- success
- output
- error
- subgoal_error
- subgoals
- execution_time

Timeout behavior

If `timeout` is provided, it is used both as the HTTP request timeout for that call and as the command timeout placed into the JSON body sent to the server.

Example

```

async with IsabelleGymAsyncClient() as client:
    session = await client.create_session(theories=["Main"])
    session_id = session["session_id"]
    lease_id = session["lease_id"]

    try:
        result = await client.execute_command(
            session_id,
            'lemma "A \<and> B \<Longrightrightarrow> B \<and> A" by blast',
            timeout=60.0,
            lease_id=lease_id,
        )

        if result["success"]:
            print("Command succeeded")
        else:
            print(result.get("error"))
    finally:
        await client.close_session(session_id, lease_id=lease_id)

```

8 Reference: Big-Step Verification

8.1 General Remarks

The big-step methods differ slightly from the incremental session methods:

- they operate on the server's `/api/v1/sessions/bigstep` endpoint
- they return the raw `httpx.Response` rather than decoded JSON
- the caller is therefore responsible for invoking `response.raise_for_status()` and `response.json()` as needed

This design makes the methods suitable for callers who need access to raw response metadata in addition to the payload.

8.2 `verify_bigstep_text`

Signature

```

async def verify_bigstep_text(
    self,
    theory_name: str,
    theory_text: str,
    *,
    field: str | None = "HOL",
    timeout: float = 300.0,
) -> httpx.Response:
    ...

```

Behavior

Submits a complete theory body for big-step verification. The request payload includes the theory name, field, theory text, and timeout. The response is returned without automatic status raising.

Important note

The server-side schema supports a `dependencies` field for big-step verification, but the current async client wrapper does not expose a `dependencies=` argument on `verify_bigstep_text`. Callers needing that server feature must extend the wrapper or issue a custom request.

Example

```
THEORY_TEXT = """
theory Scratch
imports Main
begin
lemma "A \<longrightarrow> A" by simp
end
"""

async with IsabelleGymAsyncClient() as client:
    response = await client.verify_bigstep_text(
        theory_name="Scratch",
        theory_text=THEORY_TEXT,
        timeout=300.0,
    )
    response.raise_for_status()
    result = response.json()
    print(result["success"])
    print(result.get("theory_verified"))
```

8.3 verify_bigstep_file

Signature

```
async def verify_bigstep_file(
    self,
    file_path: str | Path,
    *,
    field: str | None = "HOL",
    timeout: float = 300.0,
) -> httpx.Response:
    ...
```

Behavior

Reads a theory file from disk, derives the theory name from the file content when possible, falls back to the file stem otherwise, and then delegates to `verify_bigstep_text`.

Error behavior

If the file does not exist, the method raises `FileNotFoundError` before making any HTTP request.

Example

```

from pathlib import Path

async with IsabelleGymAsyncClient() as client:
    response = await client.verify_bigstep_file(
        Path("examples/Scratch.thy"),
        timeout=300.0,
    )
    response.raise_for_status()
    payload = response.json()
    print(payload["success"])

```

9 Recommended Usage Patterns

9.1 Pattern A: Dedicated Session for Stateful Work

Listing 2: Dedicated session pattern

```

from client.async_client import IsabelleGymAsyncClient

async def prove_once() -> None:
    async with IsabelleGymAsyncClient() as client:
        session = await client.create_session(theories=["Main"])
        session_id = session["session_id"]
        lease_id = session["lease_id"]

        try:
            await client.enter_theory(session_id, "Main", lease_id=lease_id)
            result = await client.execute_command(
                session_id,
                'lemma "A \<longrightarrow> A" by simp',
                lease_id=lease_id,
            )
            print(result)
        finally:
            await client.close_session(session_id, lease_id=lease_id)

```

9.2 Pattern B: Pooled Session for Reusable Capacity

Listing 3: Pooled session pattern

```

from client.async_client import IsabelleGymAsyncClient

async def use_pool() -> None:
    async with IsabelleGymAsyncClient() as client:
        lease = await client.acquire_session(
            theories=["Main"],
            reuse_dirty=True,
        )
        session_id = lease["session_id"]

```

```

lease_id = lease["lease_id"]

try:
    result = await client.execute_command(
        session_id,
        'lemma "True" by simp',
        lease_id=lease_id,
    )
    print(result["success"])
finally:
    await client.release_session(session_id, lease_id=lease_id)

```

9.3 Pattern C: Whole-Theory Verification

Listing 4: Whole-theory verification pattern

```

from client.async_client import IsabelleGymAsyncClient

THEORY_TEXT = """
theory Demo
imports Main
begin
lemma "A \<longrightarrow> A" by simp
end
"""

async def verify() -> None:
    async with IsabelleGymAsyncClient() as client:
        response = await client.verify_bigstep_text(
            theory_name="Demo",
            theory_text=THEORY_TEXT,
        )
        response.raise_for_status()
        print(response.json())

```

10 Error and Exception Semantics

10.1 HTTP-level failures

The methods `health`, `create_session`, `acquire_session`, `close_session`, `release_session`, `enter_theory`, and `execute_command` call `raise_for_status()` internally. They therefore propagate non-success HTTP responses as `httpx.HTTPStatusError`.

10.2 Big-step methods

The methods `verify_bigstep_text` and `verify_bigstep_file` return `httpx.Response` directly and do not call `raise_for_status()` automatically.

10.3 Transport-level failures

Connection failures, DNS resolution failures, timeout failures, and similar transport issues propagate from `httpx` using its normal exception hierarchy.

10.4 Local validation failures

`verify_bigstep_file` raises `FileNotFoundError` when the supplied path does not resolve to a file.

11 Return-Type Summary

Method	Return type	Notes
<code>health</code>	<code>dict[str, Any]</code>	Parsed JSON response.
<code>create_session</code>	<code>dict[str, Any]</code>	Parsed JSON response including <code>lease_id</code> .
<code>acquire_session</code>	<code>dict[str, Any]</code>	Parsed JSON response including <code>lease_id</code> and <code>reused</code> .
<code>close_session</code>	<code>dict[str, Any]</code>	Parsed JSON if present, otherwise empty dictionary.
<code>release_session</code>	<code>dict[str, Any]</code>	Parsed JSON if present, otherwise empty dictionary.
<code>enter_theory</code>	<code>dict[str, Any]</code>	Parsed JSON if present, otherwise empty dictionary.
<code>execute_command</code>	<code>dict[str, Any]</code>	Parsed command result.
<code>verify_bigstep_text</code>	<code>httpx.Response</code>	Caller inspects status and payload.
<code>verify_bigstep_file</code>	<code>httpx.Response</code>	Caller inspects status and payload.

12 Operational Notes

- Lease-aware methods accept `lease_id=None` in the Python signature, but leased session endpoints on the server require a valid lease header in normal usage.
- The default base URL assumes the server is available at `http://localhost:8000`.
- The module is intentionally minimal. It does not currently wrap every server endpoint available in the REST API.
- In particular, the client does not provide convenience methods for proof state inspection, history retrieval, checkpoints, rollback, or statistics, even though the server exposes such endpoints.

13 Minimal End-to-End Example

Listing 5: Minimal end-to-end session example

```
import asyncio
from client.async_client import IsabelleGymAsyncClient

async def main() -> None:
    async with IsabelleGymAsyncClient("http://localhost:8000") as client:
        session = await client.create_session(theories=["Main"])
        session_id = session["session_id"]
        lease_id = session["lease_id"]

        try:
            await client.enter_theory(session_id, "Main", lease_id=lease_id)
            result = await client.execute_command(
                session_id,
                'lemma "A \<longrightarrow> A" by simp',
                lease_id=lease_id,
            )
            print(result)
        finally:
            await client.close_session(session_id, lease_id=lease_id)

if __name__ == "__main__":
    asyncio.run(main())
```

14 Release Note for Package Consumers

As a package-facing interface, `client.async_client` should be understood as a stable usage layer over the existing REST API rather than a full SDK abstraction. It deliberately exposes the core server model—HTTP transport, lease identifiers, explicit session lifecycle, and explicit big-step response handling—without hiding backend semantics that matter to proof orchestration.